Agent Operation Authorization

https://datatracker.ietf.org/doc/html/draft-liu-agent-operation-authorization-02

Dapeng Liu / Suresh Krishnan

Problem Statement & Design Goals

Current OAuth Limitations in AI Agent Context:

1. Permission ≠ Behavior (The Injection Risk)

- **Current OAuth**: Grants broad Permissions (e.g., scope=write).
- **The Risk**: Since LLMs struggle to distinguish **Data vs. Instruction**, a **Prompt Injection** can trick the Agent into executing malicious actions that are still technically within the granted Permission.
- **Gap**: A mechanism to **bind specific behavioral policies** with OAuth authorization.

2. The Agent Delegation Challenge (Intent Decoupling)

- **Hallucination in Parsing intent:** The Agent may misinterpret the user's natural language intent.
- **Missing Contextual Binding:** Current OAuth proves who the user is, but not **which Agent is acting which user and what specific task** was authorized.
- **Gap:** Lack a cryptographic link for "Agent X acting for User Y to do Task Z".

Design Goals: Bind behavioral Policy +Agent Delegation + User Intent Evidence into the Access Token.

AOA： Workflow & Native Multi-Agent Support

```
+--------+       +----------------+       +--------+       +------------------+
|  User  |       |   AI Agent     |       |  AS    |       | Resource Server  |
+--------+       +----------------+       +--------+       +------------------+
     |                  |                      |                    |
(1)  |      prompt      |                      |                    |
     |------------------>|                     |                    |
     |                  |                      |                    |
(2)  |                  | Parse & structure    |                    |
     |                  | operation proposal   |                    |
     |                  |                      |                    |
(3)  |                  | Build operation      |                    |
     |                  | proposal JWT         |                    |
     |                  |                      |                    |
(4)  |                  | Create and sign      |                    |
     |                  | JWT with JWS         |                    |
     |                  |                      |                    |
(5)  |                  | POST /par with JWT   |                    |
     |                  |-------------------->|                    |
     |                  |                      |                    |
(6)  |                  |                      | Validate JWT       |
     |                  |                      |                    |
     |                  |                      |                    |
(7)  |                  | Return request_uri   |                    |
     |                  |<--------------------|                    |
     |                  |                      |                    |
(8)  |                  | 302 Redirect         |                    |
     |<-----------------|                      |                    |
     | /authorize?client_id=..request_uri=...  |                    |
     |---------------------------------------->|                    |
     |                  |                      |                    |
(9)  |                  |                      | Consent UI         |
     |<----------------------------------------|                    |
     |                  |                      |                    |
(10) | User approves    |                      |                    |
     |---------------------------------------->|                    |
     |                  |                      |                    |
(11) |                  |                      | Register policy    |
     |                  |                      | Issue access token |
     |                  |                      |                    |
(12) |                  | Return access token|                       |
     |                  |<--------------------|                    |
     |                  |                      |                    |
(13) |                  | Present access token                       |
     |                  |------------------------------------------>|
     |                  |                      |                    |
(14) |                  |                      |                    | Verify token
     |                  |                      |                    | Enforce policy
     |                  |                      |                    |
(15) |                  |                      |                    | Execute or deny
     |                  |                      | Response           |
     |<------------------------------------------------------------|
```

(1) User says: “Add something cheap to cart on Nov 11 night”

(2) AI Agent parses intent → builds operation proposal object

(3) Agent creates operation proposal JWT

(4) Agent creates JWT with operation proposal claim, signs with JWS

(5) Agent sends JWT to AS via PAR: POST /par { "request": "<jwt>" }

(6) AS validates JWS, extracts evidence field and validate

(7) AS issues request_uri to agent

(8) Agent redirects user to /authorize?request_uri=...

(9) User reviews the displayed operation and confirms the authorization

(10) AS register the policy and issues Agent Operation Authorization Token as access token which includes the policy_id

(11) Agent uses token to access Resource Server

(12) AS verifies user confirmation evidence

(13) RS enforces OPA policy

(14) Action executed or denied

AOA Solution: Policy-Based Authorization with AS-signed evidence binding **User + Agent + Operation**

Phase 1: The Agent_Operation_Proposal Mechanism

Core Concept:

- *A structured, machine-readable proposal generated by the Agent, serving as the **binding contract** between User, Agent, and Operation.*

1. Explicit Tripartite Binding (The "Who-What-Which" Gap Solved)

AOA explicitly cryptographically binds three elements:

- ✓ **Which User**: Identifies the Resource Owner (user identity).
- ✓ **Which Agent**: Identifies the specific Agent instance (agent identity).
- ✓ **What Specific Behavior**: Defines the exact operation policy constraints (e.g., amount <= 50, time == ”Nov. 11 night").

Agent → [Structured Proposal] → AS validates User + Agent + Intent + Policy → Signed Token → RS → [Enforces Policy]

Phase 2: Structure of Agent Operation Authorization Token

```
{
  "iss": "https://as.online-shop.example",
  "sub": "user_12345@myassistant.example",
  "aud": "https://api.online-shop.example",
  "exp": "2025-11-11T23:59:00Z",
  "iat": "2025-11-11T10:35:00Z",

  // ====== Evidence of User Confirmation ======
  "evidence": {
    "id": "evidence-confirmation-abc123",
    "user_confirmation_record": {
      "displayed_content": "Add items under $50 to cart during the Nov 11 promotion (valid until 23:59)",
      "user_action": "confirmed_via_button_click",
      "timestamp": "2025-11-11T10:33:15Z",
      "session_context": {
        "oauth_session_id": "session_abc123",
        "device_fingerprint": "dfp_xyz789"
      }
    },
    "as_signature": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...signature_over_confirmation_record"
  },

  // ====== Agent Identity ======
  "agent_identity": { ... },

  // ====== Agent Operation Authorization ======
  "agent_operation_authorization": {
    "policy_id": "opa-policy-789"
  },

  // ====== auditTrail ======
  "auditTrail": {
      "evidence_reference": "evidence-confirmation-abc123",  // References the evidence object by ID
      "semanticExpansionLevel": "medium",
      "userAcknowledgeTimestamp": "2025-11-11T10:33:00Z",
      "consentInterfaceVersion": "consent-ui-v2.1"
  },

  // ====== Optional: Reference to Proposal ======
  "references": {
    "relatedProposalId": "urn:uuid:op-proposal-456"
  }
}
```

```
{
  "evidence": {
    "type": "VerifiableCredential",
    "credentialSubject": {
      "type": "UserConfirmationEvidence",
      "userAction": "confirmed_via_ui_interaction",
      "displayedContent": "Add items under $50 to cart during Nov 11 promotion",
      "confirmationTimestamp": "2025-11-11T10:33:15Z",
      "sessionId": "session_abc123"
    },
    "issuer": "https://as.example.com",
    "issuanceDate": "2025-11-11T10:33:15Z",
    "@context": ["https://www.w3.org/2018/credentials/v1"]
  }
}
```

```
{
  "agent_identity": {
    "version": "1.0",
    "id": "urn:uuid:agent-identity-789",
    "issuer": "https://as.example.com",
    "issuedTo": "https://idp.example.com|user-12345",
    "issuedFor": {
      "platform": "personal-agent.myassistant.example",
      "client": "mobile-app-v1.myassistant.example",
      "clientInstance": "dfp_abc123"
    },
    "issuanceDate": "2025-11-11T10:35:30Z",
    "validFrom": "2025-11-11T10:35:30Z",
    "expires": "2025-11-11T23:59:00Z"
  }
}
```

auditTrail establishes a complete, semantically traceable chain—from the user's original intent to the system's final executed action—in AI Agent scenarios

Issued by AS upon user consent. Cryptographically binds Agent identity, User identity, and authorized operations.

Agent → [Structured Proposal] → AS validates User+Agent+Intent+Policy → Signed Token → RS → [Enforces Policy].

Security Model & Interoperability

Trust Boundaries:

- LLM: Untrusted (Prompt Injection/Hallucination possible).
- Agent Runtime: Trusted runtime responsible for signing requests.
- AS: The sole Witness capable of issuing evidence and extending chains.

Ecosystem Fit:

- OAuth: Uses PAR, JWT, standard error codes.
- WIMSE: Leverages Workload Identity for Agent authentication.
- OPA: Standard engine for Rego policy enforcement.

Open Source Implementation:

- **Open Agent Auth**
- IETF 125 Hackathon

Open Agent Auth

https://github.com/alibaba/open-agent-auth

Summary

Value Proposition:

- User Identify + Agent Identify + behavioral policies based Authorization
- Solves AI Hallucination/Over-privilege risks.
- Provides semantic audit trails (Human -> Agent -> RS).
- Unified model for Single & Multi-Agent orchestration.

Thanks!